DESIGN REPORT

FITNESS MEMBERSHIP MANAGEMENT SYSTEM

SESEIU24001 - Võ Châu Anh

---

1. Assumptions

These are the assumptions I used:

- Currency is VND, stored as `Double` and formatted by a `vnd` helper.
- `id` and `activationDate` are immutable (`let`) - identity never changes after registration.
- Three membership plans: Basic, Premium, Personal Training. Each has its own facilities, hours, and fee model.
- Three user roles: `member`, `staff`, `manager`. Member can only use facilities and submit change requests; staff handles day-to-day operations; manager additionally controls billing amounts and can expire/reactivate memberships.
- Monthly billing period. `billingStart` records the current period's start; `startNewBillingPeriod` resets usage counters and requires active status + zero balance.
- Temporary upgrade is only available on Basic (1…30 days), and grants Premium-level facilities and hours while active. Only one upgrade at a time.
- Promotions are pre-seeded (`Promotion.samples`) and can only be applied if valid, eligible for the plan, and not already applied.
- Check-in requires: membership usable (not suspended/expired), facility included in the plan, and time inside the access window.
- Change requests (freeze, unfreeze, trial-Premium) are submitted by the member and resolved by staff/manager. Only one pending request at a time.
- No backend - `MembershipManager(sample: true)` loads six members; `MembershipManager.preview()` reuses it for SwiftUI previews.

2. Type identification

Classes:

Class	Responsibility
Membership	Base class. Holds identity (id, activationDate), shared state (memberName, phone, status, unitFee, amountPaid, promotion, temporaryUpgradeUntil, pendingRequest, checkIns, billingStart). Owns all shared workflow: payment recording, promotion apply/remove, temporary upgrade, status change, check-in, change requests, billing period reset, and the shared authorize/ensureUsable guards.
BasicMembership	Plan-specific overrides only. Provides base facilities (gym, cardio, locker) and a 6–21 access window. Supports temporary upgrade - while active, uses PremiumMembership.perkFacilities and PremiumMembership.perkWindow. Has no usage counter, so it inherits the base no-op resetPeriodUsage().
PremiumMembership	Overrides facilities (all areas incl. pool, sauna, group studio), a wider 5–23 window, and adds group-class booking with a monthly limit. Overrides resetPeriodUsage() to clear classesUsed and bookedClasses.
PersonalTrainingMembership	Session-based plan. Tracks packageSessions, addedSessions, sessionsAttended, and enforces "sessions cannot go negative". Overrides resetPeriodUsage() to clear addedSessions and sessionsAttended.
MembershipManager	ObservableObject. Owns [Membership] and promotions (Promotion.samples). Provides search, add, aggregate totals, and the attempt(_:) wrapper that turns thrown errors into visible messages.

Structs:

Struct	Fields	Purpose
AccessWindow	let openHour: Int, let closeHour: Int; computed text, contains(_:)	Opening hours policy used for check-in validation.
Promotion	id (from code), code, title, percentOff, eligible: Set<MembershipType>, validUntil; methods isValid(on:), isEligible(for:); static samples	Discount configuration.

ChangeRequest	let id = UUID(), kind, date	One pending member request.
CheckInRecord	let id = UUID(), date, facility	One entry record.
ChargeLine	let id = UUID(), label, amount	One row on a bill.
ChargeBreakdown	lines, promotion, subtotal, discount, total, paid, outstanding	Result of the shared breakdown property, recalculated every time it is read.
InfoItem	let id = UUID(), label, value	Generic display row.

Supporting enums: MembershipType (basic/premium/personalTraining), MembershipStatus (active/suspended/expired), Facility (7 values with an icon), Permission (8 cases), UserRole (member/staff/manager, with can(_:)), RequestKind (freeze/unfreeze/trialPremium). MembershipError covers every validation path currently used.

3. Properties & Methods

Shared stored properties in Membership:

- let id, let activationDate - immutable
- private(set) var memberName, phone, status, unitFee, amountPaid, promotion, temporaryUpgradeUntil, pendingRequest, checkIns: [CheckInRecord], billingStart

Overridable policy hooks in Membership:

- type, typeName, icon, feeLabel, facilities, accessWindow, supportsTemporaryUpgrade, benefitRows, searchableText
- chargeLines() -> [ChargeLine] - main polymorphic fee calculator
- resetPeriodUsage() - no-op in base; overridden by the subclasses that track usage

Shared final members (cannot be overridden):

- breakdown: ChargeBreakdown (computed), estimatedCharges, outstanding
- authorize(_:_:action:), ensureUsable()
- updateDetails(name:phone:by:), setStatus(_:by:), updateFee(to:by:), recordPayment(_:by:)
- applyPromotion(_:by:), removePromotion(by:), grantTemporaryUpgrade(days:by:)
- startNewBillingPeriod(by:), checkIn(at:facility:by:)
- submitRequest(_:by:), resolveRequest(approve:by:)

Shared non-final members that are not overridden by any current subclass:

- hasTemporaryUpgrade, activePromotion, detailRows

Shared `private` helper:

- `applyUpgrade(days:)` - called from both `grantTemporaryUpgrade` and `resolveRequest(.trialPremium)` so the upgrade rules live in one place

Subclass-specific properties:

Class	Property	Access
`BasicMembership`	`static baseFacilities`, `static baseWindow`	`let` (constants)
`PremiumMembership`	`static perkFacilities`, `static perkWindow`	`let` (constants)
`PremiumMembership`	`monthlyClassLimit: Int`	`let`
`PremiumMembership`	`classesUsed`, `bookedClasses: [String]`	`private(set) var`
`PersonalTrainingMembership`	`packageSessions: Int`	`let`
`PersonalTrainingMembership`	`coachName`, `addedSessions`, `sessionsAttended`	`private(set) var`

Subclass-specific computed properties: `PremiumMembership.classesRemaining`, `PersonalTrainingMembership.sessionsPurchased` / `sessionsRemaining`.

Subclass-specific methods:

- `PremiumMembership.bookGroupClass(named:by:)` - enforces the monthly limit
- `PersonalTrainingMembership.attendSession(by:)` - rejects when `sessionsRemaining == 0`
- `PersonalTrainingMembership.addSessions(_:by:)` - accepts only 1…20
- `PremiumMembership.resetPeriodUsage()` and `PersonalTrainingMembership.resetPeriodUsage()` override the base no-op; `BasicMembership` keeps the no-op because it tracks no periodic usage.

4. Inheritance & Polymorphism

Hierarchy:

```
Membership
├── BasicMembership
├── PremiumMembership
└── PersonalTrainingMembership
```

`Membership` is not `final` (it must be inheritable). The three subclasses are `final`, so no deeper extension is created by accident. All shared lifecycle methods are `final` in the base, so subclasses can only override the policy hooks - this keeps billing math, promotion handling, and status transitions consistent across plans.

Inherited (written once): identity, payment recording, promotion apply/remove, temporary upgrade, status change with authorization, check-in workflow, change request workflow, `breakdown` and `outstanding` calculation.

Complete override matrix:

Member	BasicMembership	PremiumMembership	PersonalTrainingMembership
`type`	`.basic`	`.premium`	`.personalTraining`
`typeName`	"Basic" / "Basic (+ upgrade)"	"Premium"	"Personal Training"
`icon`	`figure.walk`	`star.circle.fill`	`figure.strengthtraining.traditional`
`feeLabel`	-	-	"Price per session"
`facilities`	3 base areas; Premium set while upgraded	6 perk areas (incl. pool, sauna, studio)	4 areas (gym, cardio, locker, training zone)
`accessWindow`	06:00–21:00; 05:00–23:00 while upgraded	05:00–23:00 (perk)	-
`supportsTemporaryUpgrade`	`true`	-	-
`benefitRows`	access + hours + optional upgrade line	access + hours + classes-used	coach + sessions-remaining + attended + hours
`chargeLines()`	`Monthly fee = unitFee`	`Monthly fee = unitFee`	`Package: packageSessions × unitFee` + `Added: addedSessions × unitFee`
`resetPeriodUsage()`	-	clears `classesUsed`, `bookedClasses`	clears `addedSessions`, `sessionsAttended`
`searchableText`	-	joined `bookedClasses`	`coachName`

A dash (-) means the subclass does not override that member and uses the base default. `detailRows` is defined once in `Membership` and is not overridden - plan-specific info is delivered through `benefitRows` and `chargeLines()`.

Polymorphism in practice:

- `MembershipManager.totalEstimated` sums `estimatedCharges` over every `Membership`. `estimatedCharges` uses the shared `breakdown`, which calls `chargeLines()` - each plan provides its own fee decomposition, and the manager never branches on type.
- `BillCard` and `MemberDetailView` render `breakdown.lines` and `benefitRows` generically. The only type casts are in `MemberDetailView` to attach `PremiumSection` / `TrainingSection` action forms, and in `UpgradeSection` which is conditionally shown for plans where `supportsTemporaryUpgrade == true` (only Basic today, but future plans can opt in).
- Cross-subclass dependency: `BasicMembership` uses `PremiumMembership.perkFacilities` / `PremiumMembership.perkWindow` when upgraded. This is a deliberate reuse of the Premium constants rather than duplicating them, at the cost of a small structural coupling between two concrete subclasses.

When protocol/composition would be better: if future plans can combine features (e.g., a PT plan that also includes pool access), a class tree starts to break. Protocols like `Payable`, `Bookable`, `CheckInEnabled` combined with small value types (a `UsageQuota`, a `Perks` struct) would then be cleaner. I chose class inheritance here because every plan shares the same lifecycle (`breakdown`, `recordPayment`, `checkIn`, request/approve), and the differences are entirely within the fee formula, facilities set, access window, and usage counter - all expressible as overrides.

5. Encapsulation & Validation

Access-control decisions:

Data	Access modifier	Reason
`Membership.id`, `activationDate`	`let`	Identity - must never change
`Membership.memberName`, `phone`	`private(set) var`	Only updated by `updateDetails` with trim + non-empty validation
`Membership.status`, `unitFee`, `amountPaid`, `promotion`, `temporaryUpgradeUntil`, `pendingRequest`, `billingStart`	`private(set) var`	Controlled state - changed only through a small well-defined set of mutating methods, each of which is authorized and validated first

`Membership.checkIns`	`private(set) var`	Only appended by `checkIn`
`PremiumMembership.monthlyClassLimit`, `PersonalTrainingMembership.packageSessions`	`let`	Fixed at creation
`PremiumMembership.classesUsed`, `bookedClasses`	`private(set) var`	Only mutated by `bookGroupClass` / `resetPeriodUsage`
`PersonalTrainingMembership.coachName`, `addedSessions`, `sessionsAttended`	`private(set) var`	Only mutated by their dedicated methods
`MembershipManager.memberships`	`@Published private(set)`	Views read; only the manager appends
`MembershipManager.currentRole`	`@Published` (writable)	Selected by the user
`MembershipManager.promotions`	`let = Promotion.samples`	Seed catalogue; not mutated

Controlled mutators that write `private(set)` state: `setStatus` and `resolveRequest` are the only writers of `status`; `recordPayment` and `startNewBillingPeriod` are the only writers of `amountPaid`; `applyPromotion` / `removePromotion` are the only writers of `promotion`; `submitRequest` / `resolveRequest` are the only writers of `pendingRequest`; `updateFee` is the sole writer of `unitFee`; `grantTemporaryUpgrade` / `resolveRequest` (via the private `applyUpgrade`) are the sole writers of `temporaryUpgradeUntil`; `startNewBillingPeriod` is the sole writer of `billingStart`. Every one of these methods is `final` and begins with `authorize(role, permission, action:)` before any other logic runs.

Shared guard: `ensureUsable()` is used by every operation that requires an active membership (`checkIn`, `bookGroupClass`, `attendSession`, `addSessions`, `applyUpgrade`). Suspended and expired plans are rejected through one function.

Validation rules (per method):

Rule	Where	Error
Name/phone not empty (after trim)	`updateDetails`	`emptyField`

New status ≠ current status	`setStatus`	`statusUnchanged`
Expire/reactivate requires `manageBilling`	`setStatus`	`unauthorized`
Reactivate from expired requires zero balance	`setStatus`	`cannotReactivateWithBalance`
Fee finite and > 0	`updateFee`	`invalidAmount`
Payment finite and > 0	`recordPayment`	`invalidAmount`
Payment ≤ outstanding	`recordPayment`	`overpayment`
Promotion valid + eligible + not already applied	`applyPromotion`	`promotionExpired` / `promotionNotEligible` / `promotionAlreadyApplied`
Upgrade days 1…30, plan supports it, usable, no active upgrade	`applyUpgrade`	`invalidUpgradeDays` / `upgradeNotAvailable` / `upgradeAlreadyActive`
Billing period reset: active + zero outstanding	`startNewBillingPeriod`	`periodRequiresActive` / `outstandingBalance`
Check-in: usable + facility included + inside window	`checkIn`	`membershipSuspended` / `membershipExpired` / `facilityNotIncluded` / `outsideAccessHours`
Change request: no other pending + applicable kind	`submitRequest`	`requestPending` / `requestNotApplicable` / `upgradeNotAvailable` / `upgradeAlreadyActive`
Resolve request: requires one to exist	`resolveRequest`	`noPendingRequest`

Group class: name non-empty, limit not reached	`bookGroupClass`	`emptyField` / `monthlyLimitReached`
Attend session: balance > 0	`attendSession`	`insufficientSessions`
Add sessions: 1…20	`addSessions`	`invalidSessionCount`
No duplicate ID	`addMembership`	`duplicateMembership`
Role must have permission	`authorize`	`unauthorized`

6. SwiftUI implementation

Screen structure:

- `ContentView` - `TabView` with Members and Billing. Injects `MembershipManager(sample: true)` as `@StateObject`.
- `MembersView` - search field, segmented filter by `MembershipType`, `RoleBanner`, and a list of `MembershipCard`. Cards show name, ID chip, plan pill, status pill, opening hours, and estimated charges + paid/due badge.
- `BillingView` - outstanding-balance banner, list of `BillCard` for every membership (line items + promotion discount + total/paid/outstanding).
- `MemberDetailView` - one shared layout for all plans:
  - Header (icon, name, ID, status pill) + acting-role row
  - Details (shared `detailRows`) + Edit button (`EditMemberView` sheet)
  - Plan benefits (polymorphic `benefitRows`, facilities as chips via `FlowLayout`)
  - Estimated charges (polymorphic `breakdown.lines` + promotion row + total/paid/outstanding)
  - Type-specific action forms: `PremiumSection` (group classes) or `TrainingSection` (sessions) - only the cast differs, the charges section stays polymorphic
  - `UpgradeSection` - shown only when `supportsTemporaryUpgrade`
  - `CheckInSection`, `BillingSection` (pay, change fee, apply/remove promo, start new period), `StatusSection` (change status + manage change requests)
- `EditMemberView` - sheet for name/phone only. ID and activation date are shown as read-only `LabeledContent`, matching the `let` immutability in code.

Where polymorphic behavior is visible:

- Plan cards and detail sections render different fee lines per type (Basic/Premium show one monthly-fee line; PT shows two lines when sessions were added) without any `switch` in the view - it iterates `breakdown.lines`.

- Benefit rows differ per plan without a `switch` - from `benefitRows`.
- Facilities and access window change automatically. After a temporary upgrade on Basic, the facilities chips switch to Premium areas and the card reflects it via `typeName = "Basic (+ upgrade)"`.
- Group-class and coaching controls appear only on the relevant plan (type cast). The payslip renderer above them is fully polymorphic.

7. Conclusion

The design puts shared lifecycle behavior (payment, promotion, upgrade, check-in, request/approve, status transitions, breakdown math) in `Membership`, and pushes only plan-specific variance into subclasses through eleven overridable members: `type`, `typeName`, `icon`, `feeLabel`, `facilities`, `accessWindow`, `supportsTemporaryUpgrade`, `benefitRows`, `chargeLines`, `resetPeriodUsage`, and `searchableText` - each overridden by exactly the subclass that needs a different value, and inherited by the rest. `BasicMembership` keeps the base no-op `resetPeriodUsage()` because it has no periodic usage to clear; `PremiumMembership` and `PersonalTrainingMembership` override it to reset their own counters. Sensitive state (`id`, `activationDate`, `status`, `amountPaid`, `promotion`, `unitFee`, all usage counters) is `let` or `private(set)`, and each piece of that state has a small well-defined set of `final` mutating methods that all begin with `authorize(...)` then validate. Rejected operations are explicit (`MembershipError` covers every validation path currently used) and the app surfaces them as alert messages. The result is a compact, extensible model that is easy to demonstrate in SwiftUI with clear valid/invalid feedback.